Roadmap de Refatoração do Gerenciamento de Datas

Inventario_Tools — migração para `PeriodoInventario` e `PeriodoManager`

> **Data do levantamento:** 15/10/2026
> **Escopo:** seleção de período anual, mensal e personalizado; compartilhamento do período entre telas; eliminação gradual de `app_self` como dependência global.
> **Estado avaliado:** working tree com implementação parcial não commitada.

1. Objetivo da refatoração

Centralizar o período selecionado em um objeto próprio, permitir que a tela inicial altere esse estado através de uma API explícita e notificar somente as telas que precisam reagir à mudança. O resultado esperado é que as telas não precisem acessar `app_self.data_banco_inicial`, `app_self.data_banco_final` ou os widgets de data da tela inicial.

Resultado arquitetural desejado

```
Inventario
  ├─ cria um único PeriodoManager
  ├─ conecta ações das telas
  └─ cria o Top Frame
       ├─ mantém os widgets de seleção localmente
       ├─ lê os widgets
       ├─ cria PeriodoInventario
       └─ chama PeriodoManager.atualizar_periodo(...)

PeriodoManager
  ├─ mantém o período atual
  ├─ evita notificações redundantes
  └─ notifica listeners registrados

Dashboard / Consultas / Comandos
  ├─ recebem o manager ou um snapshot do período
  └─ nunca acessam os widgets da tela inicial
```

2. Estado atual: o que já foi feito

Importante: os itens abaixo foram encontrados no working tree e representam trabalho já iniciado, mas ainda não devem ser considerados concluídos até que a integração e os testes sejam finalizados.

Item	Status	Evidência atual	Observação
Objeto de período	Parcial	`Data_Manager/PeriodoInventario.py`	Já existe dataclass imutável, validação de ordem e formatação.
Gerenciador do período	Parcial	`Data_Manager/PeriodoManager.py`	Já possui obter, atualizar, adicionar/remover listener e notificação.
Criação do manager	Parcial	`Inventario.py`	O manager é criado, porém está em variável local e ainda há código legado na classe.
Top Frame sem referências aos controles em `app_self`	Parcial	`Inv_Screen/Inv_Top_Frame.py`	Os widgets passaram a ser locais e a função recebe `periodo_manager`.
Uso do manager para alterar o período	Pendente	Não há chamada efetiva de atualização conectada aos eventos.	A função de leitura ainda não chama o manager e não há listeners integrados.
Migração das telas consumidoras	Pendente	Dashboard, Consultas e Comandos ainda usam atributos legados.	O uso de `app_self` continua em vários módulos.

3. Componentes e responsabilidades

3.1 `PeriodoInventario`

É o valor do período. Deve conter somente as datas e regras diretamente relacionadas a elas. Deve permanecer imutável depois de criado, para que uma consulta assíncrona possa usar um snapshot estável.

```
@dataclass(frozen=True)
class PeriodoInventario:
    inicio: date
    fim: date

    def __post_init__(self):
        if self.inicio > self.fim:
            raise ValueError(...)

    @property
    def inicio_banco(self) -> str: ...
    @property
    def fim_banco(self) -> str: ...
```

Decisão pendente: padronizar os nomes de formatação. O código atual usa `inicio_formatado`/`fim_formatado`, enquanto a discussão arquitetural usa `inicio_banco`/`fim_banco`. Escolher uma convenção antes da migração.

3.2 `PeriodoManager`

É o ponto único de leitura e substituição do período compartilhado. Ele não deve importar Tkinter, consultar banco ou ler widgets. A tela inicial cria o novo valor; o manager apenas o armazena e notifica.

- `obter_periodo()`: devolve o snapshot atual.
- `atualizar_periodo(novo_periodo)`: troca o valor e notifica somente quando houver mudança.
- `adicionar_listener(listener)`: registra uma função de reação.
- `remover_listener(listener)`: evita referências a telas destruídas.

4. Roadmap de execução

Fase 0 — Preparação e contrato

1. Registrar o estado atual em branch/commit ou preservar claramente as alterações locais.
2. Definir a convenção de nomes: `obter_periodo` ou `obter`; `atualizar_periodo` ou `definir`; nomes da formatação para o banco.
3. Confirmar que o formato exigido pelo Firebird continua sendo `dd.mm.yyyy`. Não misturar esse formato com o formato visual `dd/mm/yyyy`.
4. Mapear cada consumidor legado de `data_banco_inicial` e `data_banco_final`. A busca atual encontrou Dashboard, Consultas e Comandos.

Critério de conclusão: contrato documentado e lista de consumidores fechada.

Fase 1 — Consolidar o modelo de dados

1. Escolher o módulo definitivo para `PeriodoInventario` e `PeriodoManager`. A implementação atual usa `Data_Manager/`; manter essa decisão ou mover ambos para `Outros/`, evitando duas fontes de verdade.
2. Adicionar anotações de retorno e parâmetros apenas nos novos métodos, seguindo a convenção já adotada.
3. Padronizar a conversão para banco em um único local.
4. Validar entradas nulas, tipos inválidos e período invertido com erros explícitos.
5. Adicionar testes unitários para igualdade, imutabilidade, período invertido e formatação.

Não concluído: ainda não há testes específicos para as duas novas classes.

Fase 2 — Integrar o manager à inicialização

1. Criar o período padrão uma única vez no construtor de `Inventario`.
2. Salvar o manager em `self.periodo_manager`, não em variável local, porque ele será fornecido às ações que abrem outras telas.
3. Remover imports de cálculo/formatação que só existiam para as propriedades legadas.
4. Não criar managers adicionais em Dashboard, Consultas ou Comandos.

Bloqueio atual: a implementação encontrada cria `periodo_manager` como variável local, portanto o Top Frame recebe o objeto, mas a aplicação não mantém uma referência central explícita.

Fase 3 — Finalizar o Top Frame sem `app_self`

1. Corrigir a assinatura para receber somente dependências reais: `root`, `master`, `entry_list` e `periodo_manager`.
2. Manter `data_setada`, comboboxes e `DateEntry` como variáveis internas do módulo/função.
3. Implementar `obter_periodo_dos_controles()` para converter a seleção visual em `PeriodoInventario` (ou em datas que serão encapsuladas imediatamente).
4. Implementar `atualizar_periodo()`, que chama `periodo_manager.atualizar_periodo(...)`.
5. Conectar a atualização aos radio buttons, comboboxes e eventos confiáveis do `DateEntry`.
6. Executar a atualização inicial depois que os valores padrão dos controles forem definidos.
7. Preservar os atalhos F1/F2/F3/B usando `root`, sem acessar `app_self.root`.
8. Corrigir a ordem de criação dos botões: callbacks que referenciam `dashboard` não podem ser executados antes de o botão existir.

Validação: selecionar ano, mês e período personalizado deve produzir exatamente um período válido e atualizar o manager sem depender da classe `Inventario`.

Fase 4 — Separar as ações dos botões

1. Criar callbacks explícitos para abrir Consultas, Comandos, Banco, Dashboard, Configurações e Tutorial.
2. O Top Frame deve chamar callbacks, não construir telas conhecendo seus detalhes internos.
3. Para o Dashboard, o callback deve passar `periodo_manager`.
4. Para cada outra tela, passar somente as dependências que ela realmente usa.
5. Se a quantidade de callbacks ficar grande, criar uma estrutura imutável `TopFrameActions` com funções nomeadas.

Fase 5 — Migrar o Dashboard

1. Alterar `Dashboard_Screen` para receber `parent`, botão e `periodo_manager`, sem `app_self`.
2. Alterar `_recarregar_tendencia` para receber um snapshot `PeriodoInventario`.
3. Ao abrir, capturar `periodo_manager.obter_periodo()` e carregar o estado atual.
4. Registrar um listener que inicie novo carregamento quando o período mudar.
5. Remover o listener em todos os caminhos de fechamento da janela.
6. Manter a proteção contra respostas antigas usando o contador de sequência existente.
7. Garantir que cada thread receba seu próprio snapshot, nunca consultando o manager no meio da execução.

Fase 6 — Migrar Consultas e Comandos

1. Para telas abertas sob demanda, obter um snapshot no momento do clique; não é obrigatório listener.
2. Alterar funções que usam `self.data_banco_inicial/final` para receber `PeriodoInventario` ou as strings já formatadas, preferencialmente o objeto.
3. Atualizar parâmetros das consultas de Valor de Vendas, Valor de Entradas, Controla Estoque e Quantidade Exorbitante.
4. Atualizar Comandos Gerais, incluindo o comando que usa apenas a data inicial.
5. Preservar o padrão existente de `thread_execução()` e de propagação de erros.
6. Atualizar testes que usam objetos fake com `data_banco_inicial`.

Fase 7 — Remover compatibilidade legada

1. Executar busca global e confirmar zero usos de `app_self.data_banco_inicial` e

`app_self.data_banco_final`.

2. Remover `_periodo_selecionado` e as propriedades antigas de `Inventario`.
3. Remover imports mortos de `formatar_banco`, `calcular_periodo` e `MESES_REVERSO` onde não forem mais necessários.
4. Confirmar que nenhuma tela recebe a classe inteira `Inventario` apenas para acessar período.

Fase 8 — Testes, regressão e documentação

1. Testar o cálculo anual, mensal (incluindo janeiro) e personalizado.
2. Testar que período igual não notifica listeners.
3. Testar registro duplicado, remoção e notificação em cópia da lista de listeners.
4. Testar abertura/fechamento do Dashboard sem callbacks para janela destruída.
5. Executar a suíte existente e a verificação de sintaxe.
6. Testar manualmente a aplicação com troca de período durante uma consulta assíncrona.
7. Atualizar `docs/Status_Refatoracao.md` após cada fase concluída.

5. Matriz de migração por arquivo

Área	Situação atual	Ação	Resultado esperado
`Inventario.py`	Calcula o período em propriedades e cria manager local.	Guardar `self.periodo_manager`; remover propriedades ao final.	Orquestração sem ser fonte duplicada das datas.
`Inv_Top_Frame.py`	Já não grava widgets em `app_self`, mas integração está incompleta.	Conectar eventos e atualização do manager; separar ações.	Top Frame sem dependência da classe principal.
`Dashboard/Dashboard_Screen.py`	Usa `app_self.data_banco_*`.	Receber manager; usar snapshot; registrar/remover listener.	Dashboard reage a períodos novos com segurança.
`Dashboard/Dashboard_Func.py`	Recebe datas como argumentos.	Decidir contrato uniforme: objeto ou strings formatadas.	Operações recebem período consistente.
Consultas/*	Vários módulos acessam	Passar snapshot no ponto	Consultas

	atributos de `self`.	de execução.	independentes da UI principal.
`Comandos/Comandos_Gerais`	Usa `self.data_banco_inicial`.	Receber período ou data inicial explicitamente.	Comandos com dependências declaradas.
`tests/`	Há testes para cálculo antigo e fakes legados.	Adicionar testes do manager e ajustar fakes gradualmente.	Regressão protegida durante a migração.

6. Regras de implementação

- O manager é único por instância da aplicação.
- O Top Frame é a origem das alterações feitas pelo usuário.
- O manager não conhece Tkinter e não executa consultas.
- Listeners devem ser removidos quando a janela correspondente for destruída.
- Threads recebem snapshots imutáveis; nunca consultam o manager posteriormente.
- Datas internas usam `datetime.date`; formatação para o banco acontece apenas na fronteira necessária.
- Não manter duas fontes de verdade: widgets e manager não devem ser usados como estados independentes.
- Erros de período inválido devem ser explícitos; não usar fallback silencioso.
- Alterações não relacionadas à migração devem ficar fora do escopo.

7. Critérios de aceite

1. O aplicativo inicia com o período padrão correto.
2. A seleção anual atualiza o manager com 1º de janeiro a 31 de dezembro do ano escolhido.
3. A seleção mensal respeita o mês escolhido e o comportamento existente de janeiro.
4. O período personalizado normaliza ou rejeita datas invertidas conforme a regra escolhida, sem produzir estado inválido.
5. O Dashboard recarrega ao mudar o período e não atualiza widgets destruídos.
6. Consultas e comandos usam o período capturado no momento da execução.
7. Não existem consumidores de `data_banco_inicial`/`data_banco_final` fora da compatibilidade temporária, que deverá ser removida.
8. O Top Frame não recebe nem acessa `app_self`.
9. Os testes existentes continuam passando e novos testes cobrem o manager.

8. Próximas ações imediatas

1. Escolher e padronizar os nomes da API do manager e das propriedades de formatação.
2. Corrigir a criação em `Inventario.py` para usar `self.periodo_manager`.
3. Corrigir a chamada de `_create_command_buttons` para passar `root` corretamente.
4. Implementar `atualizar_periodo()` no Top Frame e conectá-la aos controles.
5. Adicionar testes unitários para `PeriodoInventario` e `PeriodoManager`.
6. Migrar primeiro o Dashboard, que é o consumidor reativo mais claro.
7. Migrar Consultas e Comandos por snapshot.
8. Executar busca final de referências legadas e remover o código antigo.

Documento de roadmap gerado a partir do estado observado no working tree em 15/10/2026. O documento descreve o plano; ele não marca como concluída uma etapa apenas porque existe código parcial.